# OBSERVACIONES DE LA PRÁCTICA

1. Larry Thomas Sanchez, lt.sanchezb1@uniandes.edu.co, 202615358

2. Jorge Claros, j.claross2@uniandes.edu.co, 202522281

## Preguntas de análisis

**1) ¿Qué relación encuentra entre el número de elementos en el árbol y la altura del árbol?**

En nuestro árbol hay 1177 elementos y la altura es 29. Si el árbol estuviera perfectamente balanceado la altura sería más o menos $\log_2(1177)$ que es cerca de 10 pero como salió casi el triple, se ve que el árbol está desbalanceado.

Esto pasa porque el BST no se balancea solo. La forma depende del orden en que llegan las llaves, si las fechas llegan casi en orden el árbol crece mucho hacia un lado, y en el peor caso queda como un single linked list con altura casi igual al número de elementos.

Entonces la relación no es fija. La altura puede ir desde log n hasta n. Eso importa porque buscar, insertar y consultar rangos cuestan O(altura), no siempre O(log n). Un árbol rojo-negro evitaría el problema porque mantiene la altura cerca de log n.

**2) Si tuviera que responder esa misma consulta y la información estuviera en tablas de hash y no en un BST, ¿cree que el tiempo de respuesta sería mayor o menor? ¿Por qué?**

Sería mayor. Una tabla de hash reparte las llaves por posiciones calculadas así que no guarda ningún orden. Eso sirve para buscar una llave exacta que es muy rápido (O(1) en promedio), pero no para pedir "todas las fechas entre A y B" como los ejemplos que se ejecutaron con el view del archivo en este caso.

Con una tabla de hash habría que revisar todas las llaves (O(n)) y quedarse con las que caen en el rango. También se podría probar cada fecha posible del rango pero eso es lento si el rango es grande.

En el BST las llaves están ordenadas. La consulta baja por el árbol se salta los subárboles que quedan fuera del rango y solo recorre las partes que sí importan. Su costo es más o menos O(altura + k), donde k es el número de elementos que están en el rango. Por eso el BST responde mejor las consultas por rango. Con solo 1177 fechas la diferencia sería pequeña pero obviamente crece con más datos.

**3) ¿Qué operación del TAD se utiliza para retornar una lista con la información encontrada en un rango de fechas?**

La operación es values(my_bst, key_initial, key_final). Recibe el árbol y las dos fechas de los límites, y devuelve una lista con los valores de todas las llaves que están entre las dos (incluyéndolas).

values es la función "máscara", (termino adoptado en clases magistrales), la cual prepara la lista vacía y llama a la función recursiva values_range, que es la que recorre el árbol. Compara los límites con la llave de cada nodo, entonces, si el límite inferior es menor baja por la izquierda, si la llave está dentro del rango

agrega el valor, si el límite superior es mayor baja por la derecha. En logic.py se usa en get_crimes_by_range, y cada valor es un nodo de fecha con su lista de crímenes.